AI Browser Agent Research & Implementation Report

---

1. Executive Summary

Problem Explored & Solution Developed

Traditional browser automation depends on brittle, hardcoded scripts that often break when websites change. This project explored whether a Large Language Model (LLM) could autonomously control a real browser using natural-language instructions, adapt to changing page states, recover from failures, and learn from past interactions. To address this, a fully agentic AI browser system was built using Playwright, LangGraph (ReAct), Groq-powered LLMs, a FastAPI backend with streaming support, and a Streamlit frontend.

Outcome

The system successfully automated tasks such as Wikipedia searches and content extraction, DuckDuckGo searches, GitHub repository navigation, and Hacker News data extraction.

---

2. Problem Statement & Objective

Problem Definition

Traditional browser automation frameworks such as Selenium and Playwright require predefined selectors and rigid execution flows. These approaches are difficult to maintain because they break whenever website structures change.

Project Objective

The objective of this research was to evaluate whether an LLM-powered agent could:

- Perform browser automation using natural language instructions
- Dynamically navigate websites without hardcoded workflows
- Recover from selector failures and learn from prior executions
- Improve efficiency over time

3. Research Summary

Existing Solutions Reviewed

1. **OpenAI Operator:** A closed, cloud-hosted browser agent platform. Although capable, it does not provide access to implementation details or custom memory mechanisms.
2. **Browser Use:** An open-source Python framework for browser control through LLMs. It served as a useful baseline but lacked advanced memory capabilities and streaming support.
3. **OpenDevin:** A comprehensive agentic operating system framework. While powerful, it was considered excessive for focused browser automation requirements.
4. **AutoGen-Based Agents:** Provided useful insights into multi-agent communication and tool-calling architectures.

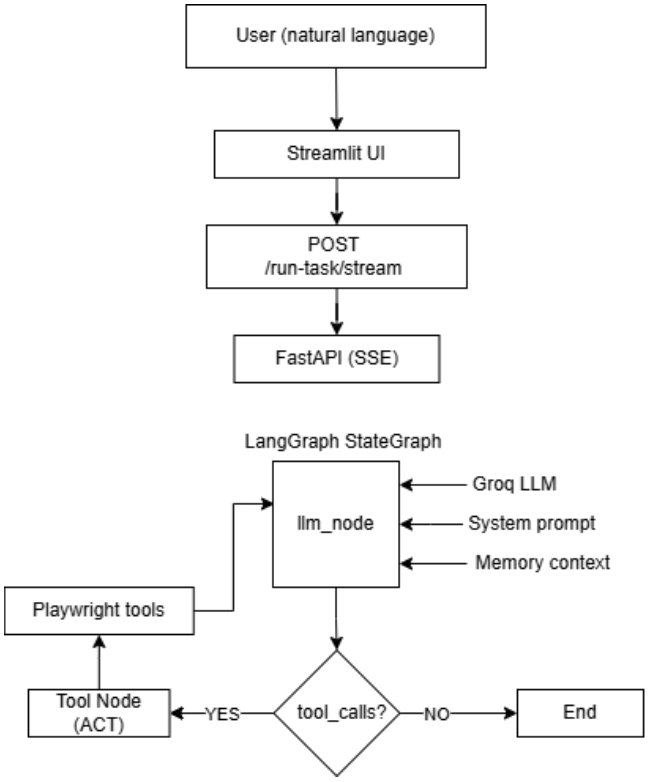
Approaches Evaluated

1. **Pure Playwright Automation:** Rejected because it relies heavily on hardcoded selectors and site-specific logic.
2. **Vision-Based Agents:** Screenshot-driven reasoning approaches were explored but deemed too slow and token-intensive for the project timeline.
3. **ReAct Architecture:** Selected as the primary approach due to its ability to combine reasoning and tool execution in an iterative loop.

Key Research Findings

- ReAct provides a reliable framework for browser automation.
- Windows environments require special handling for Playwright and asyncio compatibility.
- Raw execution logs are ineffective as memory.
- Distilled lessons improve future performance.
- LLM hallucinations occur when extraction results are empty.
- JavaScript-heavy websites require stronger page-load strategies

4. Solution Architecture



System Components

- **agent/tools.py:** Contains nine Playwright-based browser automation tools running inside a dedicated thread with a separate event loop.
- **agent/graph.py:** Implements the LangGraph StateGraph containing reasoning, execution, and routing logic.
- **agent/memory.py :**Provides both short-term and long-term memory management using session memory and ChromaDB.
- **agent/prompts.py:** Defines the system prompt, workflow instructions, and hallucination prevention mechanisms.
- **api/main.py:** Implements REST APIs, Server-Sent Event streaming, memory endpoints, and screenshot serving.
- **ui/app.py:** Provides the user interface, real-time execution logs, screenshot visualization, and memory monitoring.

5. Technology Stack

- LLM: **Groq (openai/gpt-oss-120b)**
- Agent Framework: **LangGraph**
- LLM Integration: **LangChain + langchain-groq**
- Browser Automation: **Playwright (Chromium)**
- Memory: **ChromaDB**
- API Layer: **FastAPI + Uvicorn**
- User Interface: **Streamlit**
- Package Management: **uv**

---

6. Implementation Overview

Browser Automation Layer (agent/tools.py)

Implemented nine browser automation tools: navigation, clicking, form filling, keyboard input, text extraction, scrolling, element waiting, page navigation, and screenshot capture.

Agent Execution Graph (agent/graph.py)

The agent state maintains messages, task details, execution logs, step history, and memory context. Key capabilities include rate-limit retry handling, failure detection, loop prevention, and automatic task-completion detection.

Backend Services (api/main.py)

FastAPI endpoints support task execution, streaming responses, screenshot retrieval, memory management, and health monitoring.

7. Challenges & Observations

Windows Asyncio Compatibility: Playwright requires ProactorEventLoop on Windows. Several implementation attempts failed before adopting a dedicated browser thread with its own event loop.

Hallucination During Empty Extractions: The model generated fabricated information when selectors returned no data. Multiple safeguards were implemented to prevent this behavior.

Ineffective Memory Storage: Storing raw execution logs caused repetition of failed actions. Distilled lesson extraction significantly improved memory quality.

JavaScript-Heavy Websites: GitHub and similar sites often returned incomplete page content when using domcontentloaded. Switching to networkidle improved reliability.

Groq Tool Calling Issue: Certain Groq models generated malformed tool names. Migrating to openai/gpt-oss-120b resolved the issue

---

8. Key Learnings

The project successfully demonstrated that LLM-powered browser agents can adapt to changing environments, recover from failures, learn from experience, and execute multi-step browser tasks without hardcoded scripts.

The concept is technically viable and worth further development, particularly with improvements in JavaScript-heavy site handling and action safety mechanisms.

---

9. Limitations & Future Improvements

Current Limitations

- CAPTCHA handling unavailable
- Reliability issues on JavaScript-heavy sites
- No write-action approval workflow
- Groq free-tier limitations

Future Enhancements

- **Vision-Based Recovery:** Use multimodal models when selector-based approaches fail.
- **Human Approval Layer:** Require confirmation before state-changing actions.
- **Multi-Agent Architecture:** Separate planning and execution responsibilities.
- **Parallel Execution:** Support multiple browser sessions simultaneously.
- **Session Persistence:** Resume failed tasks without restarting from the beginning.

---

10. Deliverables

Source Code

The complete project source code, documentation, and implementation artifacts are available in the project's GitHub repository:

GitHub Repository: https://github.com/likitha-111/AI-Browser-Agent

The repository contains:

- agent/ – LangGraph agent, browser tools, prompts, and memory modules
- api/ – FastAPI backend and streaming endpoints
- ui/ – Streamlit frontend application
- requirements.txt – Project dependencies
- .env.example – Environment configuration template